🛠️ PART 0: ENVIRONMENT PREPARATION

```sql
);

INSERT INTO bank_accounts (customer_name, balance) VALUES
('Alice', 1000.00),
('Bob', 500.00);

-- 2. Create a sharded table structure for Distributed DB testing
DROP TABLE IF EXISTS payment_sharded CASCADE;
CREATE TABLE payment_sharded (
    payment_id SERIAL,
    customer_id INT NOT NULL,
    amount NUMERIC(5,2),
    payment_date TIMESTAMP DEFAULT NOW()
) PARTITION BY HASH (customer_id);

-- Create 3 logical partitions (Simulating 3 different physical servers/nodes)
CREATE TABLE payment_node_1 PARTITION OF payment_sharded FOR VALUES WITH (MODULUS 3, REMAINDER 0);
CREATE TABLE payment_node_2 PARTITION OF payment_sharded FOR VALUES WITH (MODULUS 3, REMAINDER 1);
CREATE TABLE payment_node_3 PARTITION OF payment_sharded FOR VALUES WITH (MODULUS 3, REMAINDER 2);

-- Insert dummy data into the sharded environment
INSERT INTO payment_sharded (customer_id, amount)
SELECT (random() * 1000)::int, 4.99 FROM generate_series(1, 30000);
```

Data Output | Messages | Notifications

```
NOTICE:  table "bank_accounts" does not exist, skipping
NOTICE:  table "payment_sharded" does not exist, skipping
INSERT 0 30000

Query returned successfully in 145 msec.
```

PART 1: TRANSACTIONS & ACID PROPERTIES

Task 1: The "All or Nothing" Rule (Atomicity)

dvdrental/postgre... | dvdrental/postgre... | Session A* | Session B

dvdrental/postgres@PostgreSQL 17

No limit

Query | Query History

```sql
BEGIN;
-- Step 1: Add money to Bob
UPDATE bank_accounts SET balance = balance + 2000 WHERE customer_name = 'Bob';

-- Step 2: Deduct money from Alice (This will violate the CHECK constraint)
UPDATE bank_accounts SET balance = balance - 2000 WHERE customer_name = 'Alice';
```

Data Output | Messages | Notifications

```
ERROR:  new row for relation "bank_accounts" violates check constraint "bank_accounts_balance_check"
Failing row contains (1, Alice, -1000.00).

SQL state: 23514
Detail: Failing row contains (1, Alice, -1000.00).
```

dvdrental/postgre... × dvdrental/postgre... × Session A* × Session B* ×

dvdrental/postgres@PostgreSQL 17

No limit

Query Query History

```
1  SELECT * FROM bank_accounts WHERE customer_name = 'Bob';
```

Data Output Messages Notifications

	account_id [PK] integer	customer_name character varying (50)	balance numeric (10,2)
1	2	Bob	500.00

dvdrental/postgre... × dvdrental/postgre... × Session A* × Session B* ×

dvdrental/postgres@PostgreSQL 17

No limit

Query Query History

```
1  ROLLBACK;
```

Data Output Messages Notifications

```
ROLLBACK

Query returned successfully in 34 msec.
```

[ANALYSIS]

1. Why didn't Session B see Bob's new balance after Step 1? (The Isolation Property)
Even though the UPDATE for Bob was executed successfully in Session A, Session B could not see the change because of the **Isolation** property of ACID.

- **Mechanism:** Most relational databases (including PostgreSQL) use a default isolation level called **Read Committed**. This ensures that a transaction cannot see changes made by another transaction until those changes have been permanently saved using a COMMIT command.
- **Purpose:** This prevents "Dirty Reads," where a user might see temporary, unverified data that could later be cancelled. Since Session A was still "in progress" and hadn't finished its work, the database kept Bob's new balance in a private state visible only to Session A.

2. What happens to the data after the ROLLBACK? Why is Atomicity critical?

- **Data State:** After the ROLLBACK command is issued, the database engine undoes every single action performed since the BEGIN command. Bob's balance, which had been temporarily increased by $2000, is reverted to its original value ($500). The database returns to the exact state it was in before the transaction started.
- **Importance of Atomicity: Atomicity** (the "All or Nothing" rule) is the backbone of financial integrity. In a bank transfer, two things must happen: money leaves Account A and enters Account B.
    - If the system crashes or a constraint fails (like Alice not having enough funds) *after* Bob received the money but *before* Alice was charged, the bank would lose $2000 out of thin air.
    - Atomicity ensures that if any single part of the "transfer" fails, the entire operation is treated as if it never happened, preventing inconsistent balances and financial loss.

PART 2: CONCURRENCY CONTROL

Task 1: The Non-Repeatable Read Phenomenon

dvdrental/postgre... × dvdrental/postgre... × Session A* × Session B* ×

dvdrental/postgres@PostgreSQL 17

No limit

Query Query History

```
1  BEGIN; SELECT balance FROM bank_accounts WHERE customer_name = 'Alice';
```

Data Output Messages Notifications

SQL Showing rows: 1

	balance numeric (10,2)
1	1000.00

dvdrental/postgre... × dvdrental/postgre... × Session A* × Session B* ×

dvdrental/postgres@PostgreSQL 17

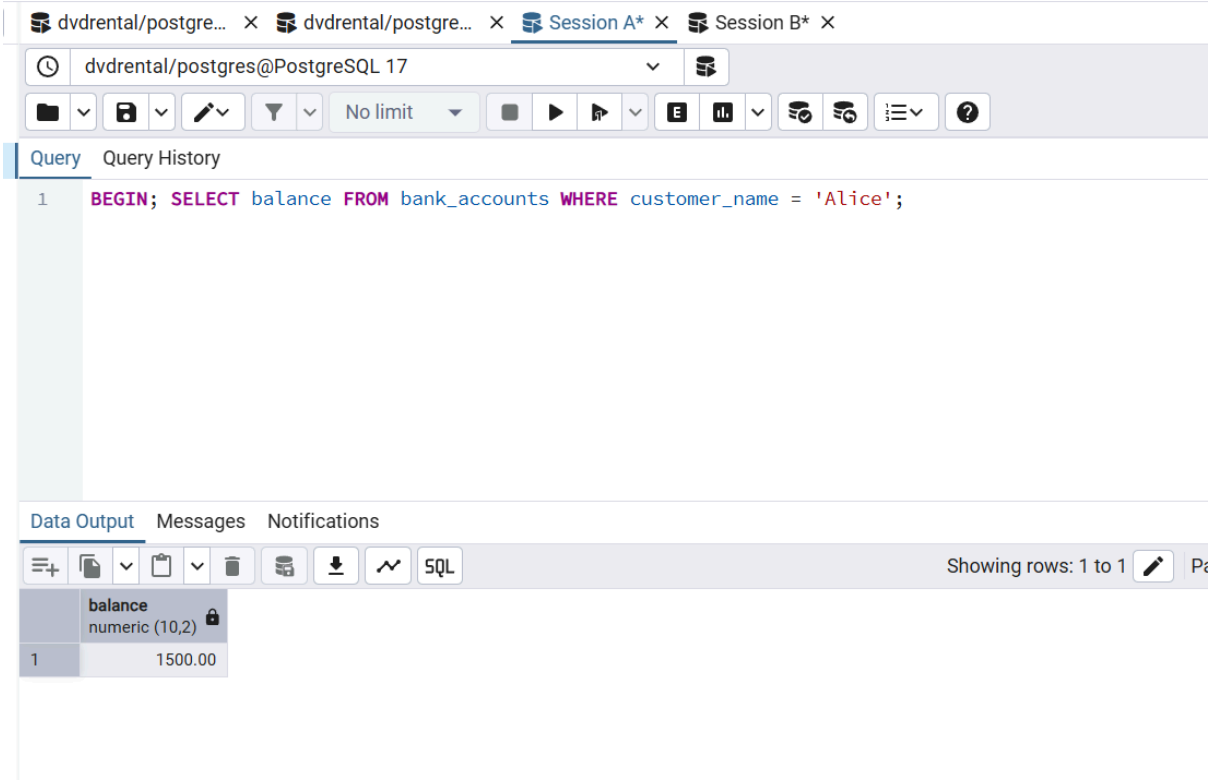
No limit

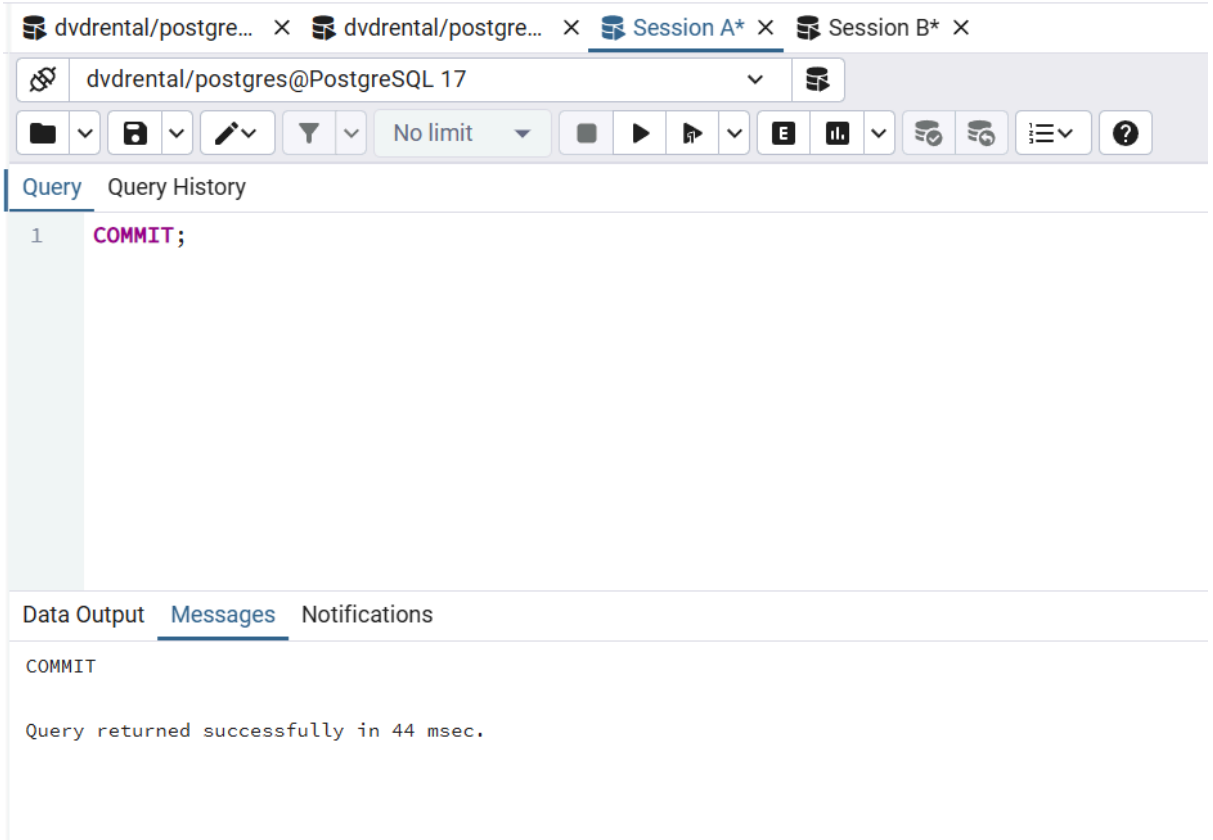
Query Query History

```
1  BEGIN; UPDATE bank_accounts SET balance = 1500 WHERE customer_name = 'Alice'; COMMIT;
```

Data Output Messages Notifications

```
COMMIT

Query returned successfully in 48 msec.
```





Đây nhé cậu, phần tiếng Anh cho **Part 2 - Task 1 (The Non-Repeatable Read Phenomenon)**. Cậu copy nguyên đoạn dưới đây paste vào Word là chuẩn luôn:

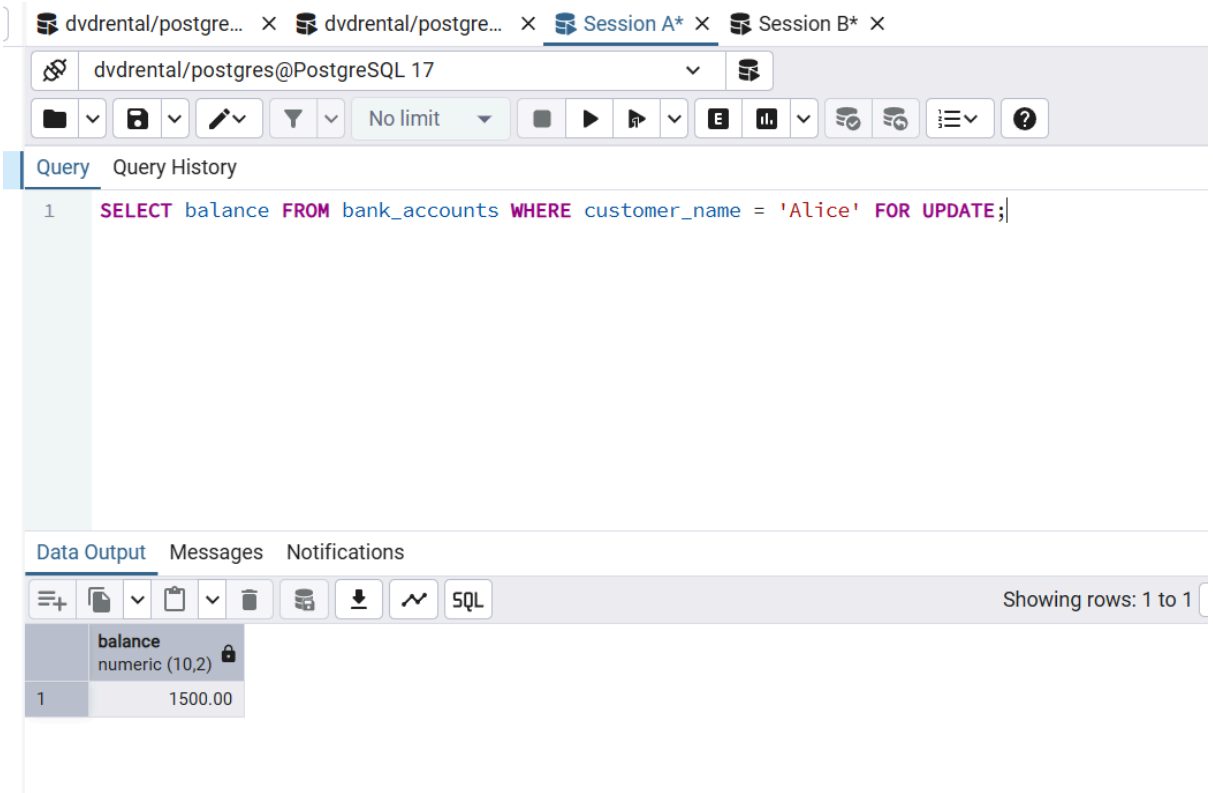
[ANALYSIS]

1. The formal name of the anomaly: The formal name of this anomaly is a **Non-Repeatable Read**. This occurs when a transaction reads the same row twice but gets different data each time. It happens because another concurrent transaction (Session B) modified and committed changes to that specific row in between the two read operations of the first transaction (Session A).

2. Rewriting the query with an Exclusive Lock: To prevent Session B from modifying the data while Session A is working with it, we must lock the row during the initial read. The rewritten query for Session A using an Exclusive Lock is:

```
SELECT balance FROM bank_accounts WHERE customer_name = 'Alice' FOR UPDATE;
```

(By appending `FOR UPDATE`, Session A places a lock on Alice's row. If Session B attempts to UPDATE this row, it will be forced to wait until Session A executes a COMMIT or ROLLBACK).



Task 2: The Deadlock Trap

dvdrental/postgre... × dvdrental/postgre... × Session A* × Session B* ×

dvdrental/postgres@PostgreSQL 17

No limit

Query Query History

```
1  BEGIN; UPDATE bank_accounts SET balance = balance + 10 WHERE customer_name = 'Alice';
```

Data Output Messages Notifications

```
UPDATE 1

Query returned successfully in 76 msec.
```

dvdrental/postgre... × dvdrental/postgre... × Session A* × Session B* ×

dvdrental/postgres@PostgreSQL 17

No limit

Query Query History

```
1  BEGIN; UPDATE bank_accounts SET balance = balance + 10 WHERE customer_name = 'Bob';
```

Data Output Messages Notifications

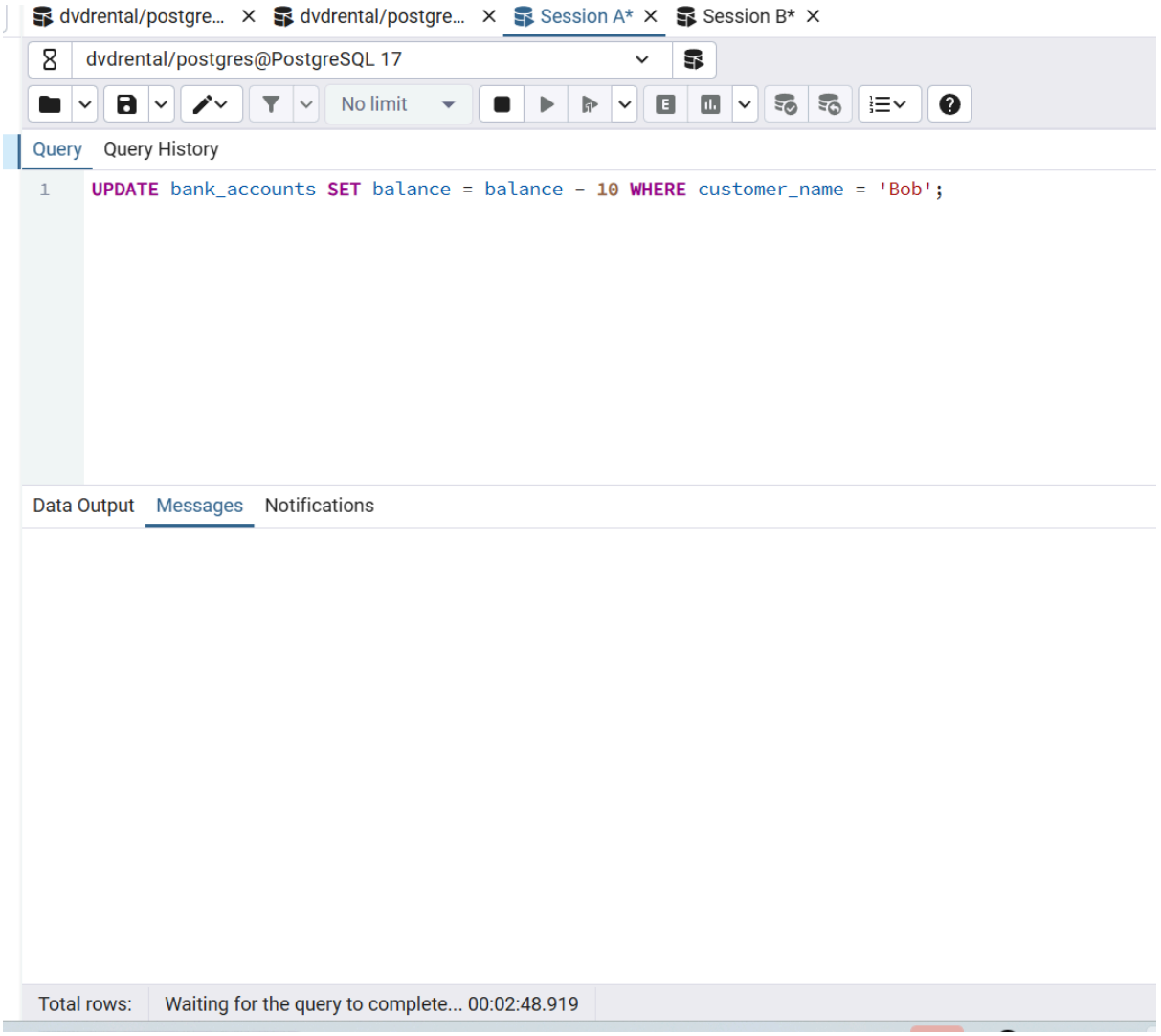
```
UPDATE 1

Query returned successfully in 34 msec.
```

Deadlock Handling in PostgreSQL

PostgreSQL does **not** try to prevent deadlocks (Prevention is too slow). Instead, it uses **Deadlock Detection**.

- When a transaction requests a lock and is blocked, it simply waits.
- After a specific timeout period (deadlock_timeout, default is usually **1 second**), Postgres wakes up and runs a cycle detection algorithm on the WFG.
- If a cycle is found, Postgres must break the cycle.

The cycle doesn't break TwT



Oh no there's the error:

dvdrental/postgre... × dvdrental/postgre... × Session A* × Session B* ×

dvdrental/postgres@PostgreSQL 17

No limit

Query Query History

```
1  UPDATE bank_accounts SET balance = balance - 10 WHERE customer_name = 'Bob';
```

Data Output Messages Notifications

```
ERROR:  current transaction is aborted, commands ignored until end of transaction block

SQL state: 25P02ERROR:  current transaction is aborted, commands ignored until end of transaction block

SQL state: 25P02
```

dvdrental/postgre... × dvdrental/postgre... × Session A* × Session B* ×

dvdrental/postgres@PostgreSQL 17

No limit

Query Query History

```
1  UPDATE bank_accounts SET balance = balance - 10 WHERE customer_name = 'Alice';
```

Data Output Messages Notifications

```
UPDATE 1

Query returned successfully in 36 msec.
```

[ANALYSIS]

1. Structural explanation of the Deadlock: A deadlock occurred in this scenario due to a classic **Circular Wait** dependency involving Exclusive Locks.

- First, Session A updated Alice's account, successfully acquiring an Exclusive Lock on Alice's row.
- Simultaneously, Session B updated Bob's account, acquiring an Exclusive Lock on Bob's row.
- Next, Session A attempted to update Bob's account, but was forced to wait because Session B held the lock.
- Finally, Session B attempted to update Alice's account, but was forced to wait because Session A held the lock. This created an infinite loop where Session A is waiting for Session B, and Session B is waiting for Session A. Neither can commit or release their locks, bringing both transactions to a standstill.

2. How PostgreSQL resolves the infinite waiting loop: PostgreSQL handles this using a **Deadlock Detection** mechanism rather than preventing it beforehand.

- When a transaction requests a lock and is blocked, it begins to wait.
- If the transaction waits longer than a specified time limit (the `deadlock_timeout` parameter, which is typically 1 second), PostgreSQL actively scans the system's Wait-For Graph (WFG).
- When the database engine detects a cycle (a closed loop of dependencies), it intervenes to break the deadlock by automatically aborting (rolling back) one of the transactions. It throws an `ERROR: deadlock detected` message to the terminated session, which releases its locks and allows the surviving transaction to successfully complete its operation.

PART 3: DATABASE SECURITY

Task 1: Principle of Least Privilege (RBAC)

dvdrental/postgres@PostgreSQL 17* × Session A* × Session B* ×

dvdrental/postgres@PostgreSQL 17

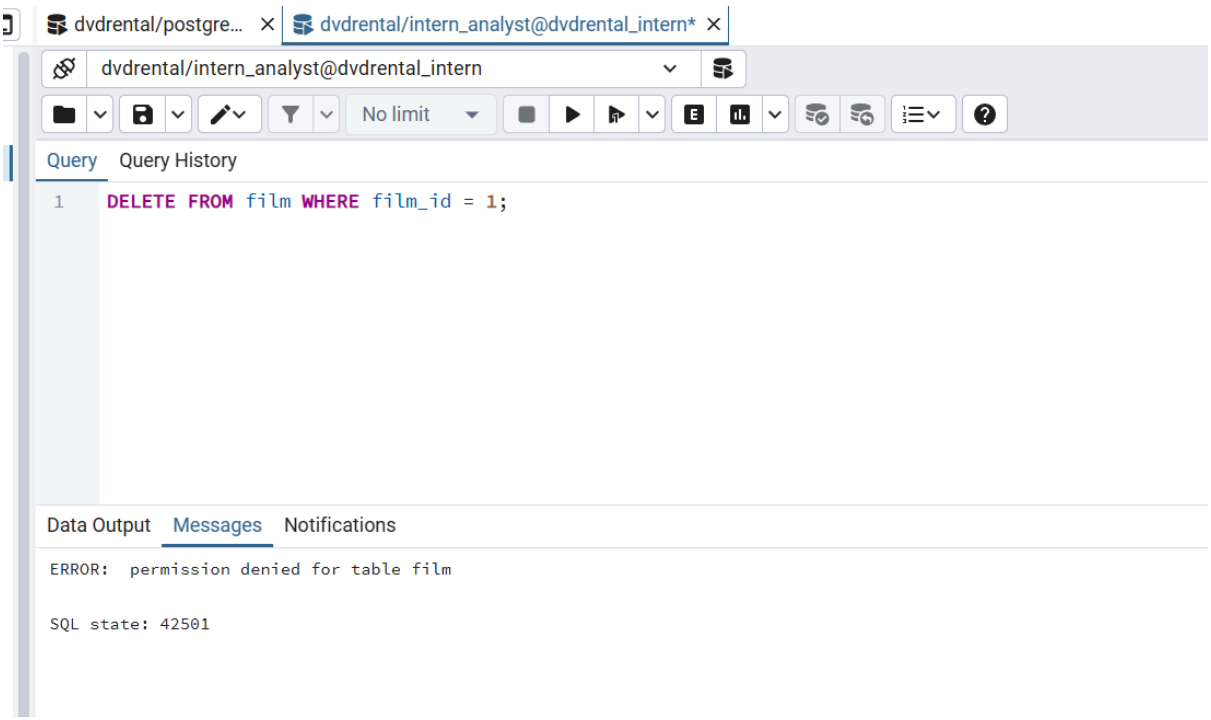
No limit

Query Query History

```
CREATE ROLE intern_analyst WITH LOGIN PASSWORD 'securepass';
GRANT CONNECT ON DATABASE dvdrental TO intern_analyst;
GRANT USAGE ON SCHEMA public TO intern_analyst;
GRANT SELECT ON film TO intern_analyst;
```

Data Output Messages Notifications

```
GRANT

Query returned successfully in 102 msec.
```



[ANALYSIS]

Why creating specific roles is superior to sharing the superuser account:

Creating specific roles (like `intern_analyst`) is a direct implementation of the **Principle of Least Privilege (PoLP)**. Sharing the default `postgres` (superuser) account among all developers is highly dangerous for several structural and security reasons:

1. **Security and Blast Radius Reduction:** The `postgres` account has unrestricted access to the entire database cluster, including the ability to drop tables, alter configurations, or delete databases. By creating a specific role, you strictly limit the "blast radius." If the intern's credentials are compromised, the attacker only gains `SELECT` access to the `film` table, preventing a catastrophic system-wide breach.
2. **Accountability and Auditing:** When multiple developers share the same superuser account, all queries executed in the database are logged under the single name `postgres`. This makes it impossible to trace who executed a specific, potentially destructive query. Using individual roles ensures that database logs accurately attribute actions to the correct user.
3. **Accident Prevention:** Developers and analysts can make unintentional mistakes (e.g., executing an `UPDATE` or `DELETE` query without a `WHERE` clause). Role-Based Access Control (RBAC) acts as a physical safeguard. If the `intern_analyst` accidentally attempts to delete a row, the database engine will block the transaction entirely, preventing accidental data loss.

Task 2: SQL Injection (SQLi) Logic

[ANALYSIS]

1. The final SQL string executed by the database:

SQL
SELECT * FROM customer WHERE email = 'x' OR 1=1 --';

2. Why this query is extremely dangerous: This is a classic SQL Injection (SQLi) attack. It is extremely dangerous because it completely bypasses the intended authentication or filtering logic.

- By injecting the single quote (`'`), the attacker closes the expected string input.
- The injected `OR 1=1` condition is a universal truth. Because of the `OR` operator, the `WHERE` clause evaluates to `TRUE` for every single row in the table, regardless of the email.
- The `--` symbol comments out the remainder of the original query (such as the closing quote and semicolon), preventing a syntax error.
- As a result, the database will return every single record in the `customer` table, leading to a massive unauthorized data breach.

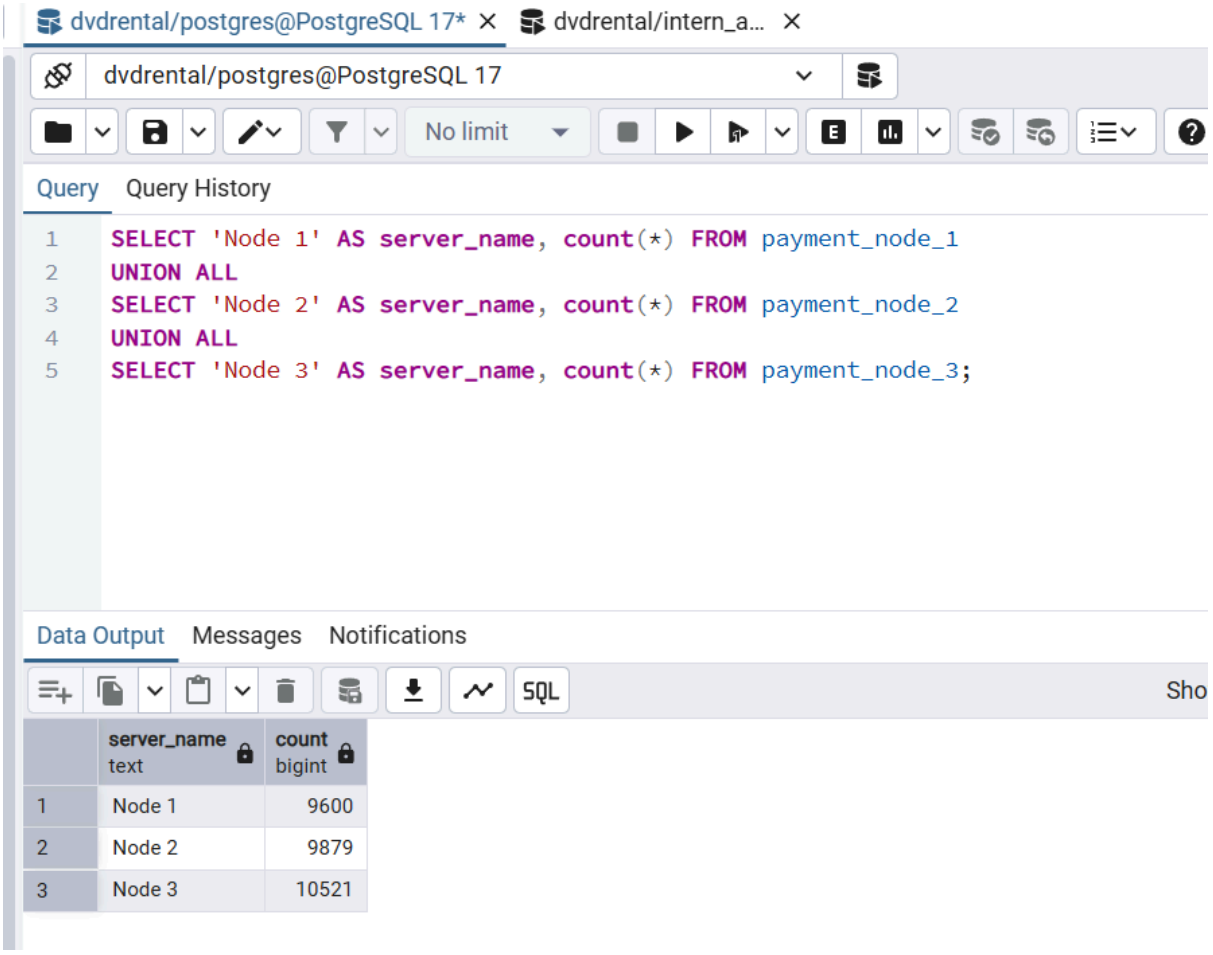
3. How Parameterized Queries neutralize this attack: Parameterized Queries (or Prepared Statements) completely neutralize SQL injection by strictly separating the SQL code structure from the user-supplied data.

- When using parameters, the database engine pre-compiles the SQL query first (recognizing the structure).
- When the user input is subsequently inserted into the parameter, the database treats it strictly as a literal value (a string), never as executable SQL commands.
- Therefore, if a user inputs `x' OR 1=1 --`, the database will simply search the table for a user whose literal email address is exactly `"x' OR 1=1 --"`. The attack string loses all its semantic power and becomes a harmless text search.

PART 4: DISTRIBUTED DATABASES & CAP THEOREM

Task 1: Horizontal Partitioning (Simulating Sharding)

1.



2. Why Vertical Scaling is not enough and why we need Sharding (Horizontal Scaling):

A single, extremely powerful server (Vertical Scaling) is fundamentally insufficient for modern, massive-scale applications like Netflix or Facebook due to several critical limitations:

- **Physical Hardware Limits:** There is a strict physical limit to how much RAM, CPU, or storage can be installed on a single machine. You simply cannot build a single server large enough to hold Facebook's petabytes of data or process Netflix's millions of concurrent video streams.
- **Single Point of Failure (SPOF):** If an application relies entirely on one massive server, any hardware failure, network issue, or routine maintenance will cause the entire global platform to go offline.
- **Exponential Cost:** Upgrading high-end enterprise hardware becomes exponentially more expensive compared to buying multiple standard servers.

Why we need Sharding (Horizontal Scaling):

Sharding (Horizontal Partitioning) solves these problems by distributing the data and the workload across multiple independent servers (nodes).

- **Limitless Scalability:** Instead of trying to build an impossibly large machine, engineers can simply add hundreds or thousands of smaller, cost-effective servers to the cluster as the user base grows.
- **Parallel Performance:** Read and write operations are distributed. When a query is executed, the system only needs to scan the specific shard holding the relevant data, drastically reducing disk I/O and query response times.
- **Fault Tolerance:** If one node fails, only a fraction of the users are affected, while the rest of the distributed database remains highly available and fully operational.

Task 2: CAP Theorem Trade-offs

[ANALYSIS]

According to the CAP Theorem, when a Network Partition (P) occurs, the system is forced to choose between Consistency (C) and Availability (A). Here is how Node 2 will respond under the different configurations:

1. If the database prioritizes Consistency (CP): Node 2 knows it cannot communicate with Node 1 to verify if its local data is up-to-date. To guarantee absolute Consistency, Node 2 will refuse to return the potentially stale data. Instead, it will respond to the user with an error message (e.g., "Service temporarily unavailable" or a timeout). It sacrifices availability to ensure no incorrect data is ever displayed.

2. If the database prioritizes Availability (AP): Node 2 is configured to always provide a response to the user, even if the data might be outdated. It will process the request and return the most recent balance it has stored locally, completely ignoring the fact that it is missing the latest transactions from Node 1. It sacrifices consistency to keep the system responsive.

3. Which approach is strictly required for a Banking application? A banking application strictly requires the **CP (Consistency)** approach. Financial systems cannot tolerate inaccurate or stale data. If an AP approach were used, a user might see an outdated balance and attempt to withdraw money they don't actually have, leading to severe issues like double-spending or massive reconciliation errors. In banking, it is always safer to temporarily deny service (lose Availability) than to operate on incorrect financial records.